

Benchmark	Base keep	Routing mem.	Main budget	Eval.	Quant.
Atari	0.50	256	$K = 2$	5 eps.	off
Panda	0.33	256	$K = 2$	20 eps.	off

TABLE V: Key TSN settings used in the main reported experiments. The base keep ratio is the per-task TSN mask keep ratio. For residual variants, ρ_Δ denotes the residual keep ratio. TSN-Aff-A-DynOcc selects ρ_Δ dynamically from $\{0.50, 0.25, 0.10\}$. Routing memory is used only for route selection and statistics, not as gradient replay. Quantization is disabled in all reported experiments.

APPENDIX A BENCHMARKS AND EXPERIMENTAL SETUP

In both benchmarks, tasks are revealed sequentially. At stage t , the learner receives only the offline dataset for the current task. Previous-task data are not replayed during optimization unless a method explicitly stores a rehearsal buffer. The TSN-based methods store compact routing memories and task statistics for route selection and latent-statistics estimation only; these memories are not used for gradient replay.

A. Atari Continual Decision Making

The Atari benchmark consists of five ALE tasks: *Breakout*, *Alien*, *Atlantis*, *Boxing*, and *Centipede*. We use expert offline trajectories stored in `expert_minari_dqn.npz` format. The environment pipeline is matched to the dataset through a Minari-like DQN preprocessing setup with frame stacking of 4, frame skip of 4, and 84×84 observations. We perform a replay-consistency check by replaying expert actions in the environment and verifying that the obtained return matches the dataset return. This check is used to ensure that the simulator and offline data are aligned.

The Decision Transformer uses a shared discrete-action interface across the task sequence. Observations have shape $(4, 84, 84)$ after preprocessing, while the action vocabulary is shared across tasks using the maximum action-space size observed in the benchmark. Main Atari results use greedy evaluation for 5 episodes per task, a maximum evaluation horizon of 27,000 environment steps, and task-specific return-conditioning targets derived from the corresponding offline expert data.

B. Panda Continual Manipulation

The Panda benchmark consists of three dense-reward manipulation tasks: *PandaReach*, *PandaPush*, and *PandaPickAndPlace*. For each task, we use an offline expert dataset. Observations are formed by concatenating `observation`, `achieved_goal`, and `desired_goal`. The dataset representation additionally includes a normalized time feature, which is matched during environment interaction through a dedicated wrapper.

Because the tasks have different raw observation and action dimensions, we map them into a shared continual-learning interface by padding all observation and action vectors to global dimensions. Shorter vectors are zero-padded in the trailing coordinates. During training, Panda action losses are computed only over the task-active action dimensions. During evaluation, the model output is truncated back to the original task-specific action dimensionality before being applied in the environment. In the reported experiments, the shared dimensions are 26 for observations and 4 for actions. Panda evaluation uses 20 episodes per task with a maximum horizon of 50 environment steps.

C. Common Implementation Settings

For Atari, cumulative replay is controlled by a replay mix of 0.5. For Panda, the cumulative baseline uses a rehearsal capacity of 5000. These replay settings are used only for replay-based baselines and are distinct from the routing memories used by TSN methods.

APPENDIX B ADDITIONAL METHOD AND METRIC DETAILS

A. TSN-NoReuse and TSN-ReplayKL

TSN-NoReuse is the no-routing sparse baseline. It keeps one model copy and disables same-copy mask overlap with previously allocated weights. Therefore, its post-hoc Reuse-all value is zero and its New fraction is 100%. This does not mean that TSN-NoReuse creates additional copies; it means that each incoming task selects new feasible parameters inside the same copy while previously allocated weights remain protected.

TSN-ReplayKL is a genuine reuse baseline rather than a no-reuse control. It uses a fixed replay-memory KL threshold τ_{KL} to decide whether an incoming task should reuse the copy of the most similar previous task or open a new copy. Inside a selected copy, mask overlap with previously allocated weights is enabled, so old weights can be reactivated by the incoming task while remaining frozen. Thus, TSN-ReplayKL differs from TSN-NoReuse, but its routing signal is observation-memory KL rather than RL-aware action or latent compatibility.

Task	Target R^*	Single-task DT
Breakout	114	112
Alien	86	62
Atlantis	1556	1505
Boxing	97	86
Centipede	135	117
NAvg (%)	88.5	

TABLE VI: Single-task Atari references. These scores separate continual-learning effects from the intrinsic difficulty of each offline Atari task.

B. Post-hoc Mask Reuse Accounting

The main text reports realized reuse statistics computed from stored binary task masks after training and evaluation. These statistics are diagnostic metrics and do not affect optimization, routing, mask selection, or rollout performance. For task t , let

$$A_{\ell, g(t)}^{(t-1)} = \bigvee_{\substack{k < t \\ g(k)=g(t)}} M_{\ell}^{(k)}$$

be the union of previous masks in the same copy. The realized same-copy reuse ratio is

$$\text{ReuseAll}(t) = \frac{\sum_{\ell} \|M_{\ell}^{(t)} \wedge A_{\ell, g(t)}^{(t-1)}\|_0}{\sum_{\ell} \|M_{\ell}^{(t)}\|_0}. \quad (22)$$

ReuseAll is measured from the perspective of the current task: it answers what fraction of the current task mask had already been allocated by any previous task in the same copy.

If a functional source task $s(t)$ is selected and reused in the same copy, we also compute source coverage:

$$\text{SourceCov}(t) = \frac{\sum_{\ell} \|M_{\ell}^{(t)} \wedge M_{\ell}^{(s(t))}\|_0}{\sum_{\ell} \|M_{\ell}^{(s(t))}\|_0}. \quad (23)$$

SourceCov is measured from the perspective of the selected source task: it answers what fraction of the chosen source subnetwork is covered by the current task mask. Therefore, ReuseAll and SourceCov describe complementary aspects of reuse. ReuseAll may be high when the current mask overlaps broadly with previous same-copy masks, while SourceCov is high only when the selected source mask is substantially preserved.

The new-mask fraction is

$$\text{New}(t) = 1 - \text{ReuseAll}(t).$$

When a method opens a fresh copy for task t , realized same-copy reuse is zero by construction because there are no previous masks in that copy. We therefore report $\text{ReuseAll}(t) = 0$, $\text{New}(t) = 100\%$, and leave SourceCov undefined unless a functional source mask is actually activated. A nearest previous task may still be recorded as routing metadata, but overlap across different copies is not counted as parameter reuse because different copies contain distinct parameter tensors.

C. Atari Normalized Average and Forgetting

For Atari, the main text reports the target-normalized average

$$\text{NAvg} = \frac{1}{N} \sum_{j=1}^N 100 \cdot \frac{P_{N,j}}{R_j^*},$$

where $P_{N,j}$ is the final score on task j after all N tasks and R_j^* is the return-conditioning target for task j . We also compute forgetting as

$$F_j = \max_{t \in \{j, \dots, N\}} P_{t,j} - P_{N,j}.$$

Under task-incremental evaluation with known task identity, low forgetting should be interpreted together with the protected-mask mechanism and the selected routing structure.

APPENDIX C SINGLE-TASK REFERENCES

Task	Mean target $\bar{R}_j$	Single-task DT	Best sweep
PandaReach	-0.208	-0.243	-0.243
PandaPush	-0.937	-0.971	-0.937
PandaPickAndPlace	-1.298	-1.027	-1.024

TABLE VII: Single-task Panda references and mean return-conditioning targets used in the corrected Panda protocol. Push and PickAndPlace remain difficult even without continual interference, which motivates reporting raw returns and MeanGap rather than normalized percentages.

Configuration	Br.↑	Al.↑	At.↑	Bo.↑	Ce.↑	NAvg↑
TSN-Aff-A, $\rho_\Delta = .10$	101	37	1551	90	86	77.6
TSN-Aff-A, $\rho_\Delta = .50$	101	37	1517	97	135	85.8
TSN-Aff-A-DynOcc	102	79	1549	97	130	95.4

TABLE VIII: Selected Atari fixed-budget affinity operating points. The fixed residual settings expose the transfer-capacity trade-off: small residuals preserve capacity but underuse early transfer, whereas large residuals improve late-task performance but can underperform on Alien. TSN-Aff-A-DynOcc selects residual size and copy allocation dynamically from copy occupancy.

Method	t_0 : Br.	t_1 : Al.	t_2 : At.	t_3 : Bo.	t_4 : Ce.
TSN-NoReuse	$c_0, -, 0.0$	$c_0, -, 0.0$	$c_0, -, 0.0$	$c_0, -, 0.0$	$c_0, -, 0.0$
TSN-ReplayKL	$c_0, -, 0.0$	$c_0 \leftarrow t_0, 57.5$	$c_0 \leftarrow t_1, 81.6$	$c_0 \leftarrow t_1, 86.7$	$c_0 \leftarrow t_1, 90.6$
TSN-Random, $\rho_\Delta = .10$	$c_0, -, 0.0$	$c_1 \leftarrow t_0, 0.0$	$c_0 \leftarrow t_0, 90.9$	$c_1 \leftarrow t_1, 90.9$	$c_1 \leftarrow t_3, 92.4$
TSN-RoundRobin, $\rho_\Delta = .10$	$c_0, -, 0.0$	$c_1 \leftarrow t_0, 0.0$	$c_0 \leftarrow t_0, 90.9$	$c_1 \leftarrow t_1, 90.9$	$c_1 \leftarrow t_1, 91.8$
TSN-Aff-A, $\rho_\Delta = .10$	$c_0, -, 0.0$	$c_1 \leftarrow t_0, 0.0$	$c_0 \leftarrow t_0, 90.9$	$c_1 \leftarrow t_1, 90.9$	$c_1 \leftarrow t_1, 91.8$
TSN-Aff-A, $\rho_\Delta = .50$	$c_0, -, 0.0$	$c_1 \leftarrow t_0, 0.0$	$c_0 \leftarrow t_0, 66.7$	$c_1 \leftarrow t_1, 66.7$	$c_1 \leftarrow t_1, 83.4$
TSN-Aff-A-DynOcc	$c_0, -, 0.0$	$c_0 \leftarrow t_0, 66.7$	$c_0 \leftarrow t_0, 90.0$	$c_1, -, 0.0$	$c_1 \leftarrow t_3, 66.7$

TABLE IX: Atari routing and realized reuse flow. Each cell reports the selected copy, the selected source task when applicable, and the percentage of the current task mask that overlaps with previously allocated masks in the same copy. Br., Al., At., Bo., and Ce. denote Breakout, Alien, Atlantis, Boxing, and Centipede. For tasks that open a new copy, any nearest source is diagnostic only and does not define realized parameter reuse.

APPENDIX D CAPACITY-CONTROLLED ATARI DIAGNOSTICS

APPENDIX E ATARI THRESHOLD AND CAPACITY SENSITIVITY

The main paper uses fixed-budget comparisons with $K = 2$ to isolate the effect of routing from the effect of additional model capacity. In this section, we report threshold and capacity sweeps as sensitivity analysis. These sweeps are useful for understanding how threshold choice and base mask density change the performance-capacity operating point, but they are not used as the primary evidence for the proposed method unless the copy budget is matched.

A. Action-affinity Fixed-budget Operating Points

Table VIII reports the regenerated capacity-controlled action-affinity operating points used to analyze the residual-allocation trade-off. We do not include incomplete broader threshold grids in the main appendix because the primary comparison is the matched-budget $K = 2$ protocol.

B. TSN-ReplayKL KL-threshold and Capacity Sensitivity

TSN-ReplayKL uses a fixed replay-memory KL threshold τ_{KL} . The main table uses the default operating point, but a threshold and keep-ratio sweep is useful to check whether the replay-memory baseline is sensitive to the chosen threshold or to the amount of newly available capacity. This comparison keeps the maximum copy budget matched to the proposed TSN-Aff-A-DynOcc variant.

Table X addresses whether the ReplayKL-style TSN baseline is limited mainly by under-allocation of new parameters. Varying the KL threshold alone does not close the gap to TSN-Aff-A-DynOcc. Increasing the base keep ratio from 0.50 to 0.55 improves TSN-ReplayKL substantially, especially on Alien and Centipede, which confirms that the amount of newly available capacity matters. However, further increasing the keep ratio overfills the reused copy and degrades the final Centipede return. This supports the motivation for occupancy-aware routing: useful transfer requires not only more capacity, but also a mechanism for controlling when and where that capacity is allocated.

Method	τ_{KL}	ρ	Copies	Br. $\uparrow$	Al. $\uparrow$	At. $\uparrow$	Bo. $\uparrow$	Ce. $\uparrow$	NAvg $\uparrow$
TSN-ReplayKL	0.10	0.50	$1/K=2$	101	46	1539	92	115	84.2
TSN-ReplayKL	0.25	0.50	$1/K=2$	101	55	1527	97	130	89.4
TSN-ReplayKL	0.50	0.50	$1/K=2$	101	52	1548	97	103	85.0
TSN-ReplayKL	1.00	0.50	$1/K=2$	101	40	1513	97	103	81.7
TSN-ReplayKL-cap.	0.25	0.55	$1/K=2$	101	79	1536	92	130	94.1
TSN-ReplayKL-cap.	0.25	0.60	$1/K=2$	101	79	1538	92	103	90.1
TSN-ReplayKL-cap.	0.25	0.70	$1/K=2$	101	79	1533	92	69	85.0
TSN-Aff-A-DynOcc	–	dyn.	2	102	79	1549	97	130	95.4

TABLE X: TSN-ReplayKL threshold and capacity sensitivity on Atari. The first block varies the fixed replay-memory KL threshold while keeping the base keep ratio fixed at $\rho = 0.50$. The second block increases the base keep ratio at the default $\tau_{KL} = 0.25$. Increasing ρ to 0.55 substantially improves TSN-ReplayKL, but larger values overflow the single reused copy and reduce Centipede performance. TSN-Aff-A-DynOcc is shown for reference and achieves the strongest normalized average without increasing the global base keep ratio.

Configuration	Run	Copies	Reach $\uparrow$	Push $\uparrow$	Pick $\uparrow$	AvgGap $\downarrow$	AvgFJ $\downarrow$
TSN-ReplayKL	r1	2	-7.865	-1.026	-2.061	3.505	3.838
TSN-ReplayKL	r2	2	-14.716	-1.203	-1.433	5.638	7.270
TSN-ReplayKL	r3	2	-10.546	-1.083	-1.585	4.259	5.179
TSN-Aff-A ($\tau_A = 0.65$)	r1	3	-0.188	-0.956	-1.505	0.737	0.000
TSN-Aff-A ($\tau_A = 0.65$)	r2	2	-1.482	-0.949	-2.643	1.546	0.647
TSN-Aff-A ($\tau_A = 0.45$)	r1	3	-0.189	-0.972	-1.422	0.715	0.001
TSN-Aff-A ($\tau_A = 0.35$)	r1	3	-0.189	-0.972	-1.422	0.715	0.001
TSN-Aff-L ($\tau_L = 25$)	r1	3	-0.189	-0.949	-1.308	0.670	0.000
TSN-Aff-L ($\tau_L = 25$)	r2	3	-0.188	-0.956	-1.505	0.737	0.000
TSN-Aff-H ($\tau_H = 0.50, \alpha = 0.70$)	r1	2	-1.482	-0.949	-2.643	1.546	0.647

TABLE XI: Auxiliary Panda threshold and reuse runs. These runs are retained as sensitivity analysis. They show that replay-memory KL routing is weaker and less stable than affinity-based routing in heterogeneous continuous-control transfer. The main Panda comparison uses the corrected fixed-budget results in Table III.

Method	Pick route	Pick reuse $\uparrow$	Pick return $\uparrow$
TSN-ReplayKL	new copy	0.0	-1.607
TSN-Aff-L-NoWarm	Reach copy	59.0	-1.510
TSN-Aff-L residual diag.	Reach copy	82.5	-1.629

TABLE XII: PandaPickAndPlace routing diagnostic. The table reports the route used for the hardest Panda task, realized same-copy mask overlap, and final PickAndPlace return. The diagnostic illustrates that TSN-ReplayKL can allocate a new copy and therefore obtain zero realized reuse for PickAndPlace, whereas latent-affinity variants can induce measurable source-mask reuse. These diagnostic rows are not intended to replace the main TSN-Aff-L-Flex result in Table III.

APPENDIX F

PANDA AUXILIARY AND CAPACITY-SENSITIVITY RUNS

We retain the Panda threshold experiments as auxiliary sensitivity analysis. In addition, we report a TSN-ReplayKL capacity-sensitivity check analogous to the Atari analysis. The goal is to test whether the replay-memory KL baseline is limited mainly by under-allocation of new parameters, or whether its weakness on Panda is primarily due to the observation-memory routing signal. The main Panda comparison remains the fixed-budget table in the main text.

A. Auxiliary Panda Threshold Runs

B. PandaPickAndPlace Routing Diagnostic

C. TSN-ReplayKL Capacity and KL-threshold Sensitivity

Table XIII shows that the Panda behavior differs from Atari. On Atari, increasing the TSN-ReplayKL base keep ratio substantially improves the baseline, although it still remains below TSN-Aff-A-DynOcc. On Panda, increasing the keep ratio does not produce the same benefit. The best capacity-sensitive TSN-ReplayKL row improves PickAndPlace relative to weaker sensitivity runs, but it does so while degrading PandaPush. Conversely, using a loose KL threshold increases realized same-copy reuse for PandaPickAndPlace, but the final PickAndPlace return remains poor. This supports the main Panda conclusion: in heterogeneous continuous-control transfer, replay-memory KL routing is not a sufficient compatibility signal, and latent-affinity routing with flexible adaptation provides a better fixed-budget trade-off.

Method	τ_{KL}	ρ	Copies	Reach $\uparrow$	Push $\uparrow$	Pick $\uparrow$	Gap $\downarrow$	Pick reuse $\uparrow$
ReplayKL, main	—	0.33	2	-0.188	-1.209	-1.607	0.200	0.0
ReplayKL, sens.	0.10	0.33	2	-0.187	-0.959	-2.399	0.381	39.5
ReplayKL, sens.	0.25	0.33	2	-0.187	-0.937	-2.816	0.513	0.0
ReplayKL, sens.	1.00	0.33	1/ $K=2$	-0.187	-0.937	-2.241	0.321	60.7
ReplayKL, sens.	0.25	0.50	2	-0.187	-1.477	-1.477	0.246	0.0
ReplayKL, sens.	0.25	0.67	2	-0.187	-1.552	-1.458	0.265	0.0
TSN-Aff-L-Flex	—	—	2	-0.189	-0.933	-1.315	0.013	—

TABLE XIII: Panda TSN-ReplayKL capacity and KL-threshold sensitivity under the fixed copy budget $K = 2$. Gap is computed with the same task-specific mean-return targets as in the main Panda protocol. Pick reuse is the realized same-copy mask overlap for PandaPickAndPlace. Increasing the TSN-ReplayKL keep ratio improves PickAndPlace relative to weaker sensitivity runs, but it also degrades PandaPush. A looser KL threshold increases same-copy reuse but still leaves PickAndPlace substantially worse than TSN-Aff-L-Flex.

APPENDIX G

PANDA PADDING AND SHARED TASK INTERFACE

The Panda tasks considered in this work differ in their raw observations and action dimensionalities. To support a single continual-learning backbone across the whole task sequence, we embed all tasks into a shared observation space of dimension 26 and a shared action space of dimension 4. For observations, task-specific state vectors are first formed by concatenating `observation`, `achieved_goal`, and `desired_goal`, along with an additional normalized time feature used in our environment wrapper. If the resulting vector is shorter than the global observation dimension, it is zero-padded in the trailing coordinates.

For actions, task-specific action vectors are analogously embedded into the shared global action space by zero-padding trailing coordinates. During training, padded action dimensions are excluded from the loss by means of a task-specific binary action mask. Concretely, if task t has an active action mask $m_a^{(t)} \in \{0, 1\}^4$, then only dimensions with $m_a^{(t)} = 1$ contribute to the Panda regression objective. During evaluation, the predicted action vector is truncated back to the original task-specific action dimensionality before being applied in the environment. This design allows all Panda tasks to share a single Decision Transformer backbone while avoiding artificial penalties on padded coordinates.

APPENDIX H

NOTES ON LEARNING CURVES

The previous version of the appendix included qualitative learning curves for representative fixed-threshold operating points. We omit these figures from the current version because the main comparison has been reframed around capacity-controlled $K = 2$ routing, and the final Atari variant is TSN-Aff-A-DynOcc. If learning curves are included in a later version, they should be regenerated for the current main methods, namely TSN-NoReuse, TSN-ReplayKL, TSN-Aff-A-DynOcc, and optionally Cumulative, using the same evaluation protocol as the main tables.